

Agenda :

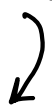
- Power Function — 2 variations
  - ✓ • Print array
  - ✓ • Indices of an Array
  - ✓ • Check palindrome
- 

Q1 Power function.

a

n

→ 0



a<sup>n</sup>

$$a = 2, n = 5 \longrightarrow \begin{matrix} 2^5 \\ \underline{32} \end{matrix}$$

$$a = 2, n = 4 \longrightarrow 2^4 = 16 \quad \}$$

$$a = 3, n = 4 \longrightarrow 3^4 = 81$$

Variation 1 :

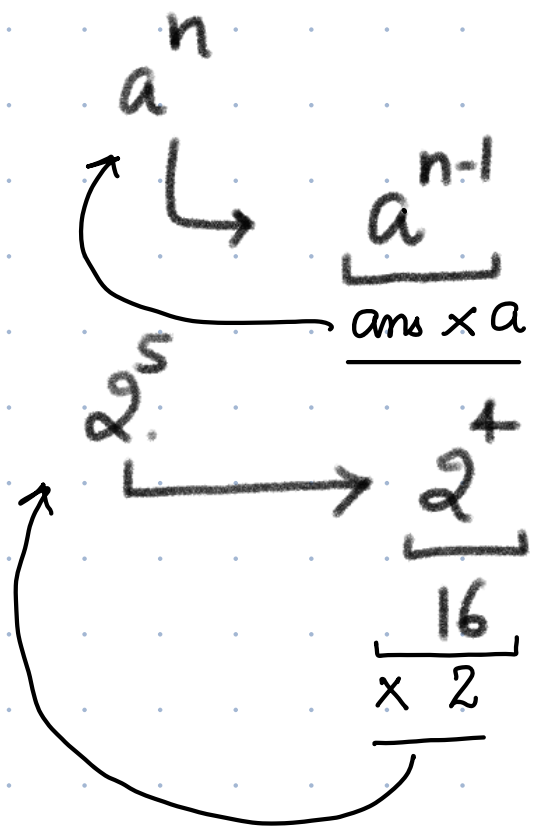
- 1) Function definition (P) ✓
- 2) Recursive Case (SP) ✓
- 3) Base Case

```
int power(int a, int n) {
```

```
    int ans = power(a, n-1);
```

```
    return ans * a;
```

```
}
```



Close my eyes



Dry Run - with 1 example

$a = 2, n = 5$

```
int power(int a, int n) {
    if (n == 0) { return 1; }
```

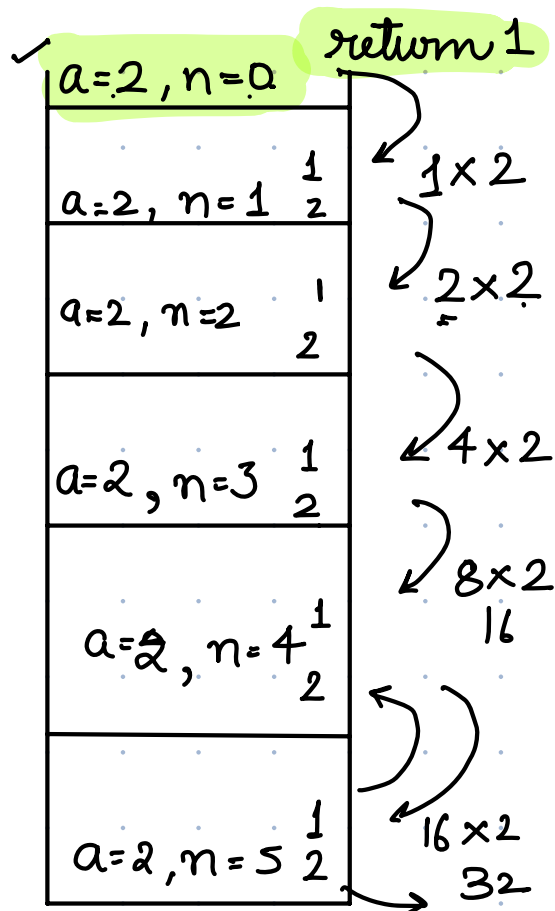
```
    int ans = power(a, n-1);
```

```
    return ans * a;
```

```
}
```

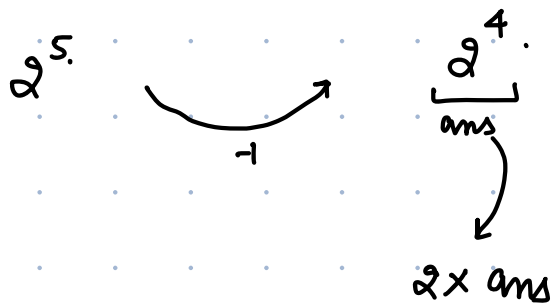


TC: Number of RC \* TC of 1 RC  
 $O(N)$  \*  $O(1)$



P  
SP  $\longrightarrow$  from ans of SP

P ✓



$$\underline{3^6} = 3^3 * 3^3$$

$\searrow$   
 $\underline{3^3}$   
 $\underline{27} * \underline{27}$

```
int power ( int a, int n) {
```

```
    int ans = power ( a,  $\frac{n}{2}$  );
```

```
    return ans * ans;
```

```
}
```

$$3^6 = 27 * 27 \quad 3^3 = 27 \quad 3^3 = 27$$

$$P = \begin{array}{|c|c|} \hline a & n \\ \hline 3 & 6 \\ \hline \end{array}$$

1 level down

$$SP: \begin{array}{|c|c|} \hline a & n/2 \\ \hline 3 & 3 \\ \hline \end{array} = \textcircled{\underline{27}}$$

ans \* ans

$$\begin{array}{|c|c|} \hline \text{odd} \downarrow \\ \hline a & n \\ \hline 3 & 7 \\ \hline \end{array}$$

↙

$$n/2$$

$$7/2 = 3$$

$$3^3 = 27$$

$$3^7$$

$$\begin{array}{c} 27 * 27 * 3 \\ 3^3 * 3^3 * 3 \end{array}$$

```
int power ( int a, int n) {
```

```
    int ans = power ( a, n/2 );
```

```
    if ( n % 2 != 0 )
```

```
        return ans * ans * a;
```

```
    else
```

```
        return ans * ans;
```

```
}
```

```

int power ( int a, int n) {
    ✓ if (n == 0) { return 1; }
    int ans = power (a, n/2);

```

```

    if (n % 2 != 0)
        return ans * ans * a;

```

```

    else
        return ans * ans;
}

```

a=3  
n=8

|          |   |
|----------|---|
| a=3, n=0 |   |
| a=3, n=1 | 1 |
| a=3, n=2 | 1 |
| a=3, n=4 | 1 |
| a=3, n=8 | 1 |

ans=1

ans  
3

ans  
9

ans  
81

ans \* ans  
81 \* 81  
6561

a=5, n=1

|          |   |
|----------|---|
| a=5, n=0 |   |
| a=5, n=1 | 1 |

1 ans

ans \* ans \* a  
1 \* 1 \* 5

5 ✓

5

```

int power ( int a, int n) {
    ✓ if (n == 0) { return 1; }
    int ans = power (a, n/2);

```

```

    if (n % 2 != 0)
        return ans * ans * a;

```

```

    else
        return ans * ans;
}

```

TC = Number of Recursive calls × TC of 1 Recursive call

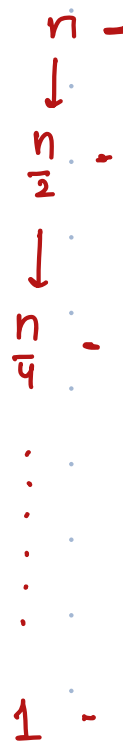
$\log_2 n$  \*  $O(1)$

=  $O(\log_2 n)$

$$a=2, n=16$$



$$a=2, n=37$$



$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \dots, \quad 1$$



$$\log_2 n$$

$$P = a^n$$

SP

```

    graph TD
      a_n_minus_1[a^{n-1}] -- "X*" --> a_n[a^n]
      a_n --> P
  
```

$$P = a^n$$

$$SP = \underbrace{a^{n/2}}_X$$

even:  $X * X$

odd:  $X * X * a$

$$P = a^n$$

$$SP = \underbrace{a^{n/3}}_{ans}$$

# Recursion in Arrays ] MB

P ✓

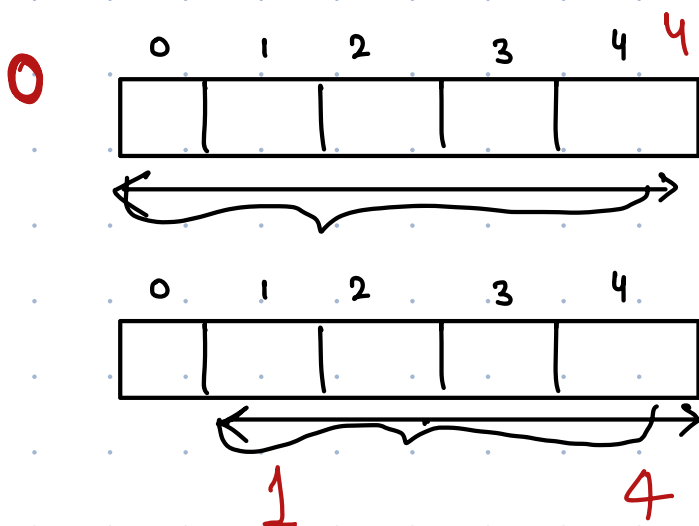
SP ✓

Dry Run + Base Case

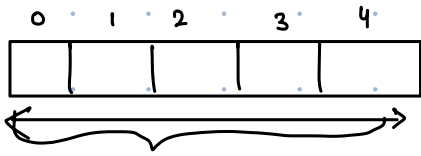
P: arr → <sup>SP</sup> i to N-1

SP: arr → i+1 to N-1

} index pointer



Print an array, using recursion



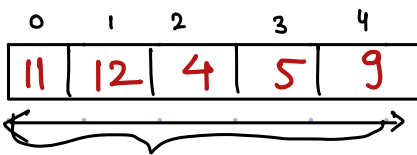
void

printArray (int [] A, int i) {

print (A[i]);

printArray (A, i+1);

}



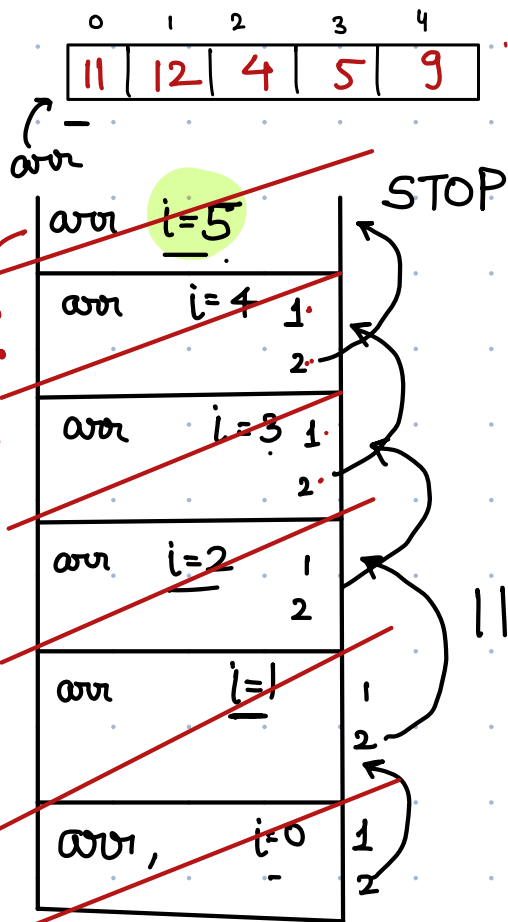
0 - end

✓ 11

1 - end

12 4 5 9

```
void printArray (int[] A, int i) {
    if (i == arr.length) { return; }
    print (A[i]);           1
    printArray (A, i+1);    2
}
```



Complete code

```
✓ void printArray (int[] A) {
    printArray (A, 0);
}
```

```
void printArray (int[] A, int i) {
    if (i == arr.length) { return; }
    print (A[i]);           1
    printArray (A, i+1);    2
}
```

}

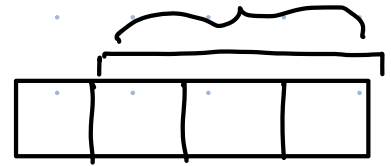
To find the sum of the array using recursion, what is the Recursive state to be used ?

Tip: We have used `max(arr[index], findMax(arr, index + 1))` to find the maximum of the array.

29 users have participated

|                                  |   |                                                                    |     |
|----------------------------------|---|--------------------------------------------------------------------|-----|
| <input checked="" type="radio"/> | A | <code>arr[index] + findSum(arr, index + 1)</code>                  | 59% |
| <input type="radio"/>            | B | <code>max(arr[index], findSum(arr, index + 1))</code>              | 21% |
| <input type="radio"/>            | C | <code>arr[index] + max(arr[index], findSum(arr, index + 1))</code> | 17% |
| <input type="radio"/>            | D | <code>arr[index] - findSum(arr, index + 1)</code>                  | 3%  |

P = sum of entire array



$$SP = \left\{ \begin{array}{l} \text{sum}(arr, i+1) \\ + \\ arr[i] \end{array} \right.$$

### Example

B = 5

Input:

$$A = [4, 5, 3, 1, 5, 4, 5]$$

$$B = 5$$



Output:

[1, 4, 6]

Q3.

Find all the Indices at which B occurs in the given array, A:

A = [1, 2, 3, 1, 1]  
B = 1

3

| 0 | 1 | 2 |
|---|---|---|
| 0 | 3 | 4 |
| ↑ | ↑ | ↑ |

$$A[i] == B$$

[0, 3, 4]

```

int[] allIndices (int[] A, int B, int i, int c) {
    if (i == A.length) {
        return;
    }
    if (A[i] == B) {
        allIndices(A, B, i+1, c+1);
        arr[c] = i;
    } else {
        allIndices(A, B, i+1, c);
    }
}

```

$$A = [1^0, 2^1, 3^2, 1^3, 1^4]$$

$$B = 1$$

| arr | 0 | 1 | 2 |
|-----|---|---|---|
|     | 0 | 3 | 4 |

$$\underline{arr[c] = i}$$

STOP

Match ✓

Match ✓

X

X

Match ✓

|          |  |
|----------|--|
| i=5, c=3 |  |
| A B      |  |
| i=4, c=2 |  |
| A B      |  |
| i=3, c=1 |  |
| A B      |  |
| i=2, c=1 |  |
| A B      |  |
| i=1, c=1 |  |
| A B      |  |
| i=0, c=0 |  |
| A B      |  |

arr  
arr[2]=4  
arr[1]=3

X

X

arr[0]=0

Using Arrays →

int []

allIndices

(int[] A, int B, int i, int c) {

if (i == A.length) {

int[] arr = new int[c];

return arr;

}

int[] res;

if (A[i] == B) {

res = allIndices(A, B, i+1, c+1);

res[c] = i;

} else {

res = allIndices(A, B, i+1, c);

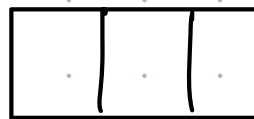
}

}

arr

res

2042



✓ ArrayList / vector / list

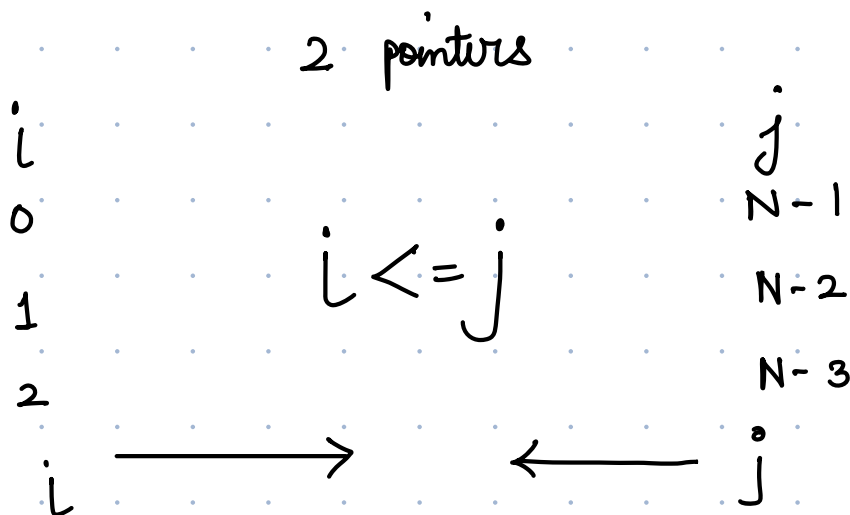
## Q4. Palindrome

String  $\rightarrow$  radar  
Yes

$\checkmark \underset{\checkmark}{a} \underset{\checkmark}{d} \underset{\checkmark}{a} \checkmark = \checkmark \underset{\checkmark}{r} \underset{\checkmark}{o} \underset{\checkmark}{p} \underset{\checkmark}{o} \checkmark$   
Yes

"chanan" — No

"nitin" — Yes

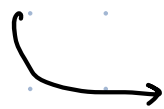


```
boolean isPalindrome (string str) {  
}
```

Dry Run  
↓  
Base Case

```
boolean isPalindrome (string str, int0 i, intN-1 j) {  
    HW  
    if (str[i] == str[j]) {  
        boolean ans = isPalindrome(str, i+1, j-1)  
        return ans  
    }  
    else {  
        return false  
    }  
}
```

i . . . . . j  
1 2 3 4 5  
n a v e e n



naveen, 1, 4  
i j

No  
false

es  
true

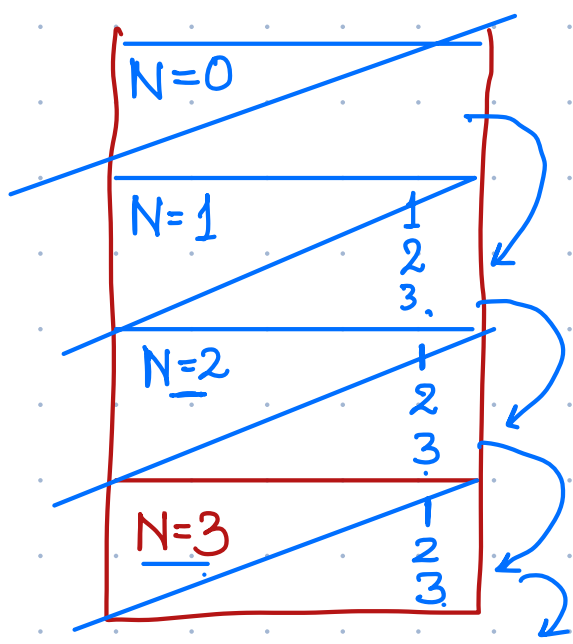
n i t i n  
- - - -  
[ ]

i t i  
-----  
Yes

What is the output of the following code for N = 3?

```
void solve(int N){  
    if(N == 0)  
        return;  
    1 print(N);  
    2 solve(N-1);  
    3 print(N);  
}
```

3 2 1 1 2 3



What is the output of the following code for N = -3?

```
void solve(int N){  
    if(N == 0)  
        return;  
    print(N); 1  
    solve(N-1); 2  
}
```

N = -3

-3 -4 -5 -6

|        |        |
|--------|--------|
| N = -7 |        |
| N = -6 | 1<br>2 |
| N = -5 | 1<br>2 |
| N = -4 | 1<br>2 |
| N = -3 | 1<br>2 |

STOP?

Stack  
overflow

Doubts —

1)  $\frac{2^{-5}}{2^5} = \frac{1}{2^5}$

$\frac{2^{-5}}{2^5} = \frac{1}{2^5}$

$\frac{1}{2^5} = \frac{1}{32}$  → decimal

Python :

min heap  $\longrightarrow$   $O(1)$

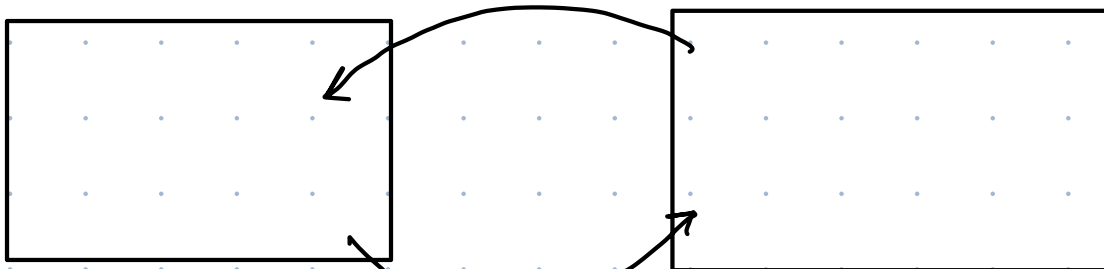
max heap  $\longrightarrow$  -ve

(56)

$\longleftarrow$   $\frac{-56}{\text{---}}$

$$B = -5$$
$$\longleftarrow -1 * B = 5$$

---



f1

f2

Output

$\searrow$   
patience